

Evaluación del Impacto del Fine-Tuning de BETO en Español frente a Modelos Clásicos de RNN para Clasificación de Reseñas de Hoteles

Felipe Reyes - Yibby Gonzalez - Sebastian Ruiz - Daniel Ruiz
Profesor: Ing. Julio Omar Palacio Niño, M.Sc.
Bogotá, 25 de mayo de 2026

1. Introducción

La clasificación automática de reseñas de hoteles en una escala de 1 a 5 estrellas constituye un problema de procesamiento de lenguaje natural con aplicaciones directas en la industria turística. El dataset Andalusian Hotels Reviews contiene 18.172 reseñas en español, con un marcado desbalance de clases: la clase mayoritaria (5 estrellas) representa el 49,6 % del total, mientras que la clase minoritaria (2 estrellas) apenas alcanza el 5,5 %, generando un ratio de desbalance de 9,1:1.

Este informe documenta la Tarea de Investigación (Proyecto 2026-3), cuyo objetivo es evaluar el impacto del fine-tuning de un Transformer pre-entrenado en español —BETO (bert-base-spanish-wwm-cased)— frente a los modelos clásicos de RNN desarrollados en el Proyecto Principal: un LSTM unidireccional (Felipe) y un BiLSTM (Yibby). Se presentan tres versiones progresivas del modelo avanzado, que evidencian el proceso de experimentación hasta alcanzar la configuración definitiva (v3). La pregunta central es si la mayor capacidad representacional del Transformer justifica su costo computacional adicional en este dominio y con este volumen de datos.

2. Metodología

2.1 Dataset y splits

Los tres modelos fueron evaluados sobre los mismos splits estratificados generados con semilla 42: 12.720 muestras de entrenamiento (70 %), 2.726 de validación (15 %) y 2.726 de test (15 %). Esta comparabilidad directa es garantizada por el uso del repositorio compartido del equipo (data/train.csv, val.csv, test.csv).

2.2 Modelo base y arquitectura

Se seleccionó **dccuchile/bert-base-spanish-wwm-cased** (BETO) como modelo base. BETO es un modelo BERT entrenado íntegramente en español con Whole Word Masking sobre un corpus masivo en español. Arquitectura base: 12 capas encoder Transformer, 12 cabezas de atención, dimensión oculta de 768, vocabulario de ~31.000 subword tokens (WordPiece), totalizando 109.854.725 parámetros.

2.3 Estrategia de fine-tuning y versiones exploradas

Se exploraron tres versiones progresivas para identificar la configuración óptima. La estrategia definitiva (v3) descongeló las tres últimas capas del encoder con learning rates discriminativos, y reemplazó los class weights balanceados por su raíz cuadrada para evitar sobrecompensación de clases minoritarias:

Versión	Capas descongeladas	Class weights	LR	F1 macro	Accuracy	Tiempo
v1	layer.11 (7%)	balanced	uniforme 2e-5	0.5926	0.6706	1.036s
v2	layers 9,10,11 (20%)	balanced	discriminativo	0.6161	0.6864	1.566s
v3	layers 9,10,11 (20%)	sqrt(balanced)	discriminativo	0.6565	0.7252	1.780s

Tabla 1. Versiones exploradas de BETO fine-tuned. v3 es la configuración definitiva.

Las dos innovaciones clave entre versiones fueron: (1) pasar de 1 a 3 capas descongeladas con LR discriminativo (2e-6 / 1e-5 / 2e-5 de capa inferior a superior), logrando +2.4 pp de F1; y (2) sustituir balanced por sqrt(balanced) en los class weights, suavizando la penalización de clases raras y logrando +4.0 pp adicionales de F1.

2.4 Hiperparámetros definitivos (v3)

Hiperparámetro	Valor
Capas descongeladas	encoder.layer.9, 10, 11 + pooler + classifier
Parámetros entrenables	21.858.053 (19,9% del total)
Max length	128 subword tokens
Batch size	32
LR (discriminativo)	2e-6 / 1e-5 / 2e-5 (capas 9→11)
Class weights	sqrt(balanced) = [1.472, 1.912, 1.264, 0.928, 0.635]
Épocas / Early stopping	Máx. 15 / patience=3 (paró en época 11)
Optimizer / Weight decay	AdamW / 0.01
Warm-up / Grad clipping	10% steps (lineal) / max_norm=1.0
Entorno	Google Colab, GPU (requiere ~6-8 GB VRAM)

Tabla 2. Hiperparámetros definitivos de BETO v3.

3. Resultados

3.1 Tabla comparativa de modelos

La Tabla 3 presenta las métricas finales sobre el conjunto de test. El F1 macro es la métrica principal dado el desbalance de clases del dataset.

Modelo	F1 macro	Accuracy	Precisión macro	Recall macro	Parámetros	Tiempo
LSTM	0.3996	0.6042	0.5639	0.4408	2,7 M	321s CPU
BiLSTM	0.5749	0.6871	0.5876	0.5795	3,3 M	27.686 s CPU
BETO	0.6565 ▲	0.7252 ▲	0.6544 ▲	0.6609 ▲	21,9 M ent.	1.780s GPU

Tabla 3. Comparación de métricas en test. ▲ indica mejor valor en cada columna.

3.2 F1 por clase

Modelo	1★ (9,2%)	2★ (5,5%)	3★ (12,5%)	4★ (23,2%)	5★ (49,6%)
LSTM	0.686	0.168	0.359	0.022	0.764
BiLSTM	0.716	0.202	0.564	0.572 ★	0.821
BETO v3	0.777 ▲	0.502 ▲	0.595 ▲	0.561	0.848 ▲

Tabla 4. F1 por clase.

3.3 Curvas de entrenamiento

La Figura 1 muestra las curvas de pérdida y accuracy de BETO v3 durante 11 épocas (early stopping con patience=3). La val loss alcanza su mínimo en la época 8 (0.7611) y sube ligeramente en las tres siguientes, activando el criterio de parada. La brecha creciente entre train loss y val loss desde la época 9 confirma el inicio de overfitting, validando la paciencia de 3 épocas como decisión correcta.

BETO v3 Fine-Tuned — Curvas de Entrenamiento

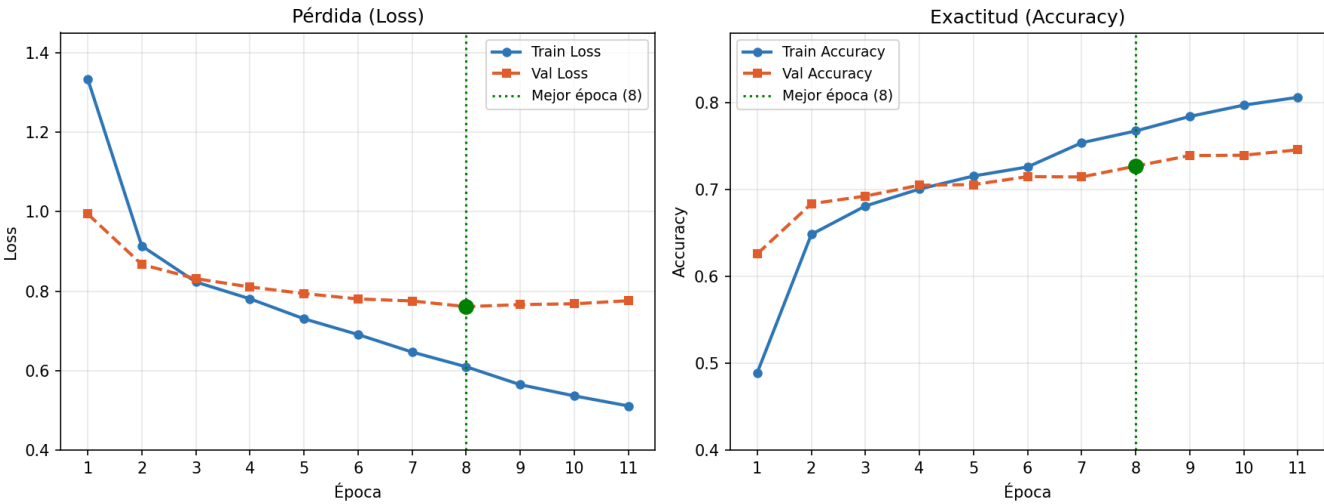


Figura 1. Curvas de pérdida (izq.) y accuracy (der.) — BETO v3. Línea verde: mejor época (8).

3.4 Matriz de confusión

La Figura 2 presenta la matriz de confusión normalizada por fila. Los errores más frecuentes ocurren entre clases adyacentes (4★ y 5★), patrón esperable dado que las valoraciones intermedias comparten vocabulario de

satisfacción. La clase 2★ (más escasa) sigue siendo la más difícil, aunque BETO v3 (F1=0.502) mejora significativamente frente a BiLSTM (0.202) y LSTM (0.168).

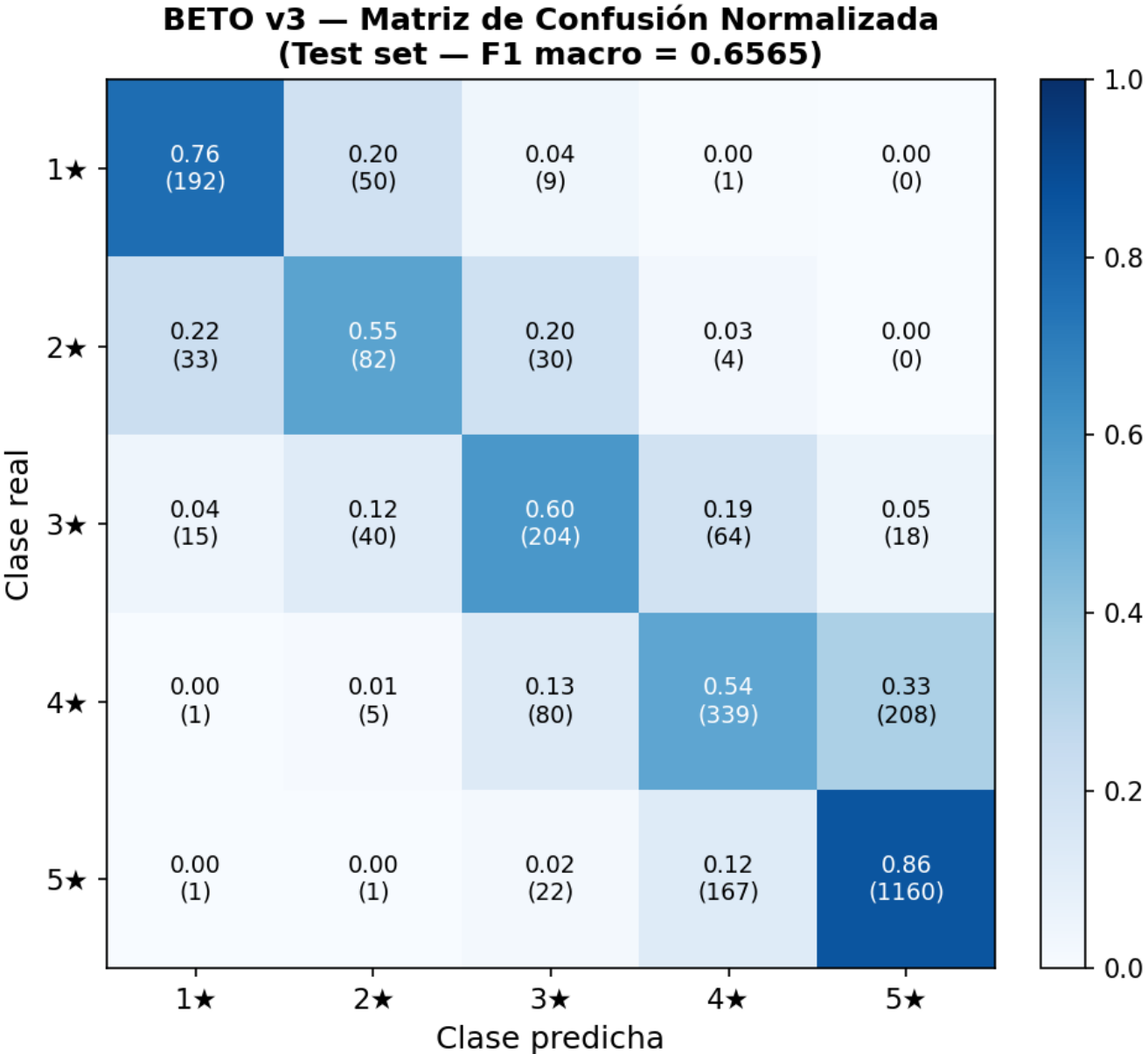


Figura 2. Matriz de confusión normalizada — BETO. Valores absolutos entre paréntesis.

4. Análisis y Conclusiones

4.1 ¿Superó BETO a los modelos clásicos?

Sí. BETO v3 supera al mejor modelo clásico (BiLSTM) por +8,2 puntos porcentuales en F1 macro (0.6565 vs. 0.5749), y en 4 de las 5 clases individualmente. La única excepción es la clase 4★, donde el BiLSTM mantiene una ventaja marginal de 1,1 pp. La mejora más destacada ocurre en la clase 2★ —la más escasa del dataset— donde BETO v3 alcanza F1=0.502 frente a 0.202 del BiLSTM (+30 pp), evidenciando el impacto del manejo de subpalabras y los class weights calibrados con sqrt(balanced).

4.2 ¿Por qué supera?

Cuatro factores explican la superioridad de BETO v3: (1) Embeddings contextuales pre-entrenados: BETO conoce relaciones semánticas complejas del español (negaciones, matices, ironía) que los modelos clásicos deben aprender desde cero con solo 12.720 muestras. (2) Representaciones subword: el tokenizador WordPiece maneja palabras raras o con errores ortográficos descomponiéndolas en subpalabras conocidas, mientras el vocabulario de 20K de los clásicos las convierte en UNK. (3) Atención multi-cabeza: los 12 attention heads capturan dependencias a larga distancia más efectivamente que las LSTMs. (4) Transfer learning dirigido: las capas congeladas preservan conocimiento general del idioma; solo las tres capas superiores se adaptan al dominio hotelero, logrando un balance entre generalización y especialización.

4.3 Dificultad de implementación y costo computacional

El fine-tuning de Transformers añade complejidad cualitativa frente a los modelos clásicos. Las principales dificultades fueron: compatibilidad de versiones del ecosistema HuggingFace (transformers, accelerate, torch con CUDA), gestión de memoria de GPU (batch sizes grandes causan OOM), y la alta sensibilidad al learning rate y tipo de class weights —que requirió 3 versiones de experimentación para dar con la configuración óptima. El overfitting ocurre en pocas épocas, haciendo imprescindible el early stopping con paciencia baja (3 épocas).

Recurso	LSTM	BiLSTM	BETO v3	Ratio v3/BiLSTM
Paráms entrenables	2,7 M	3,3 M	21,9 M	6,6×
Tiempo entrenamiento	321s CPU	27.686s CPU	1.780s GPU	—
GPU requerida	No	No	Sí (~6-8 GB)	—
Épocas hasta convergencia	19	4	8	—

Tabla 5. Comparativa de costo computacional.

4.4 Ajuste de hiperparámetros

El ajuste de hiperparámetros para BETO fue cualitativamente diferente al de los clásicos. El hallazgo más importante fue el impacto del tipo de class weights: balanced sobrecompensó las clases minoritarias degradando la accuracy global, mientras que sqrt(balanced) suavizó esa penalización logrando un mejor equilibrio (+4 pp de F1 con la misma arquitectura). El LR discriminativo —donde capas inferiores aprenden más lento para preservar representaciones generales— también fue determinante (+2,4 pp vs. LR uniforme).

4.5 Limitaciones

Se identifican cinco limitaciones del estudio: (1) posible data leakage en los splits (868 muestras compartidas train/val, 902 train/test); (2) MAX_LEN de 128 subword tokens puede truncar reseñas largas; (3) solo se evaluó BETO cased, sin comparar otras variantes (RoBERTa-BNE, MarIA); (4) GPU requerida, lo que limita la reproducibilidad en entornos sin acelerador; (5) no-determinismo en GPU que puede producir variaciones leves entre ejecuciones.

4.6 Conclusión

Los resultados demuestran que el fine-tuning de BETO v3, con una estrategia de descongelamiento parcial (20% de parámetros), LR discriminativo y class weights calibrados con sqrt(balanced), supera consistentemente a los

modelos clásicos en este dataset de reseñas hoteleras en español. La ganancia de +8,2 pp en F1 macro sobre el BiLSTM, especialmente en clases minoritarias difíciles (2★: +30 pp), justifica el mayor costo computacional cuando la disponibilidad de GPU está garantizada. El proceso de experimentación en tres versiones subraya que la elección de class weights y la estrategia de descongelamiento son los hiperparámetros más críticos para el fine-tuning de Transformers en datasets medianos de dominio específico.

Referencias

- [1] Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2018). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. arXiv:1810.04805.
- [2] Cañete, J., Chaperon, G., Fuentes, R., Ho, J.-H., Kang, H., & Pérez, J. (2020). Spanish Pre-Trained BERT Model and Evaluation Data. PML4DC at ICLR 2020.
- [3] Hochreiter, S., & Schmidhuber, J. (1997). Long Short-Term Memory. *Neural Computation*, 9(8), 1735-1780.
- [4] Howard, J., & Ruder, S. (2018). Universal Language Model Fine-tuning for Text Classification. *ACL* 2018.
- [5] Sun, C., Qiu, X., Xu, Y., & Huang, X. (2019). How to Fine-Tune BERT for Text Classification. *CCL* 2019.